EECS 3311 Software Design
Winter 2026 Sections Z
Deliverable 1
Team 14
February 28, 2026

Git Repository (Public): https://github.com/gvik3739/SoftwareDesign_Project.git Part I: Requirements Eliciting and Modeling

1. **User (Primary Actor – General)** [Req1, Req3, Req4, Req8, Req9, Req10]

The User actor represents all registered individuals who interact with the system to reserve laboratory equipment.

This actor is modeled using actor generalization, since multiple user types share identical use cases but differ in business rules (pricing, identification requirements, payment options). Common Interactions:

- Register Account
- Login
- Provide ID / Certification
- Pay Deposit
- Reserve Equipment
- Extend Reservation
- Cancel Reservation
- Modify Reservation

The following actors are modeled as specializations of the User actor (generalization relationship in UML):

1.1. Student (Specialized Actor)
- Pays $10/hour
- Must provide Student ID
- May use institutional account
- No departmental approval required

1.2. Faculty (Specialized Actor)
- Pays $15/hour
- Must provide Staff ID
- Requires departmental approval

1.3. Researcher (Specialized Actor)
- Pays $20/hour
- Must provide certification number
- May use research grants
- Requires departmental approval

1.4. Guest (Specialized Actor)
- Pays $30/hour
- Must provide certification number
- No departmental approval required

2. **Lab Manager (Primary Actor)** [Req6, Req7]

Interacts with:

- Equipment Management
- Equipment Activation/Deactivation
- Maintenance Scheduling

Notes / Constraints:

- Can add new equipment
- Can enable/disable equipment
- Can mark equipment unavailable for maintenance
- Account created exclusively by Head Lab Coordinator

3. **Head Lab Coordinator (Primary Actor)** [Req2]

Interacts with:

- Lab Manager Account Creation Notes / Constraints:
- The only actor authorized to auto-generate Lab

Manager accounts

4. **Equipment Sensor System (Secondary Actor – Automated)** [Req5]

Interacts with:

- Equipment Monitoring
- Status Updates

Notes / Constraints:

- IoT-based sensors
- Tracks equipment usage
- Sends operational data to the system

5. **System Clock (Secondary Actor – Automated)** [Req4, Req9]

Interacts with:

- Deposit Timeout Monitoring
- Reservation Start Time Tracking
- Late Arrival Detection Notes / Constraints:
- Tracks reservation start times automatically
- T riggers deposit forfeiture if user does not arrive within 20 minutes
- Ensures accurate timing for booking start, extension, and cancellation rules
- Represents an automated system time service used by the platform

6. **Payment Gateway (Secondary Actor – External System)** [Req4, Req10]

Interacts with:

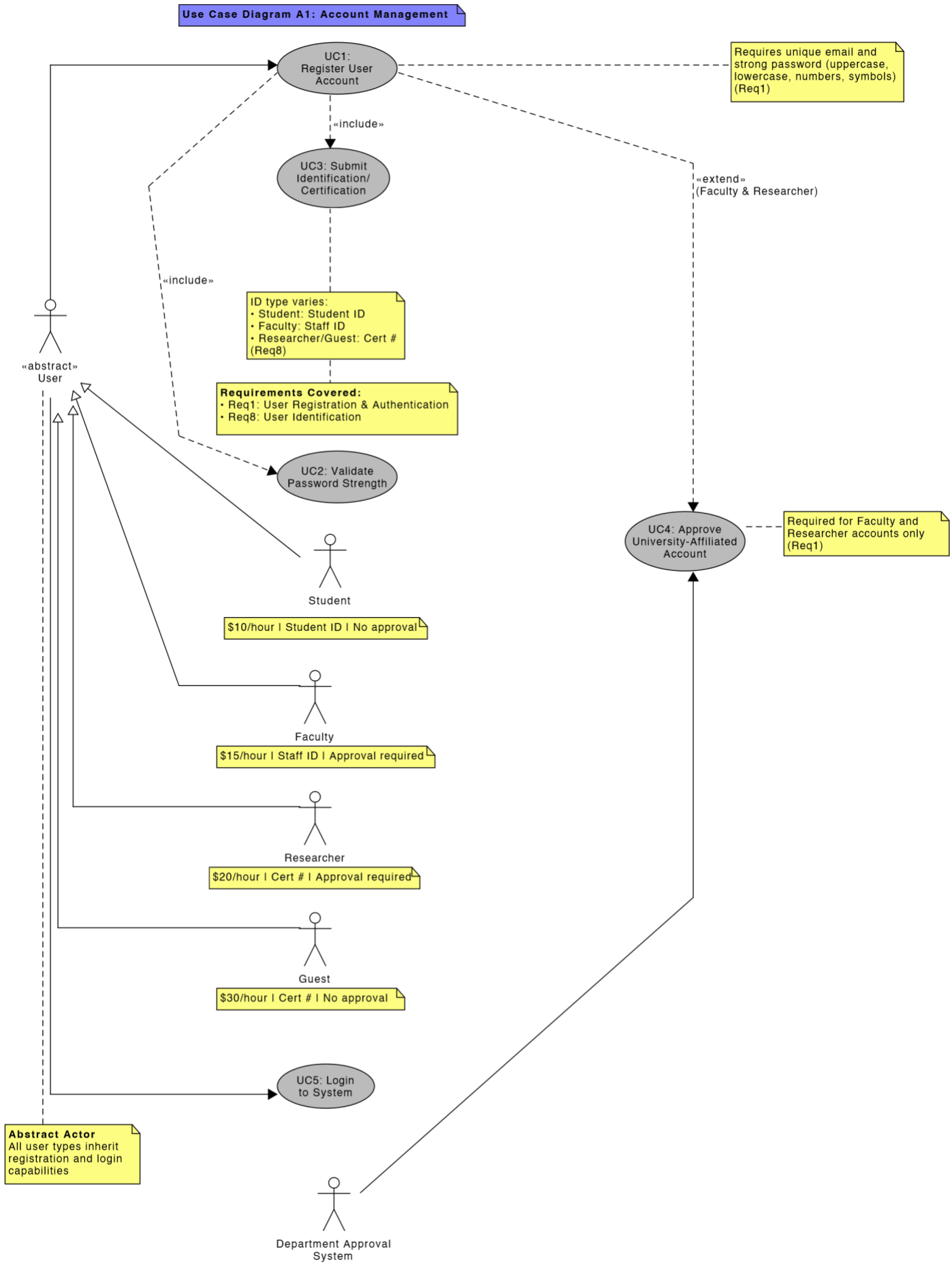
- Process Payment
- Refund Deposit Notes / Constraints:
- Handles credit card payments
- Handles debit payments

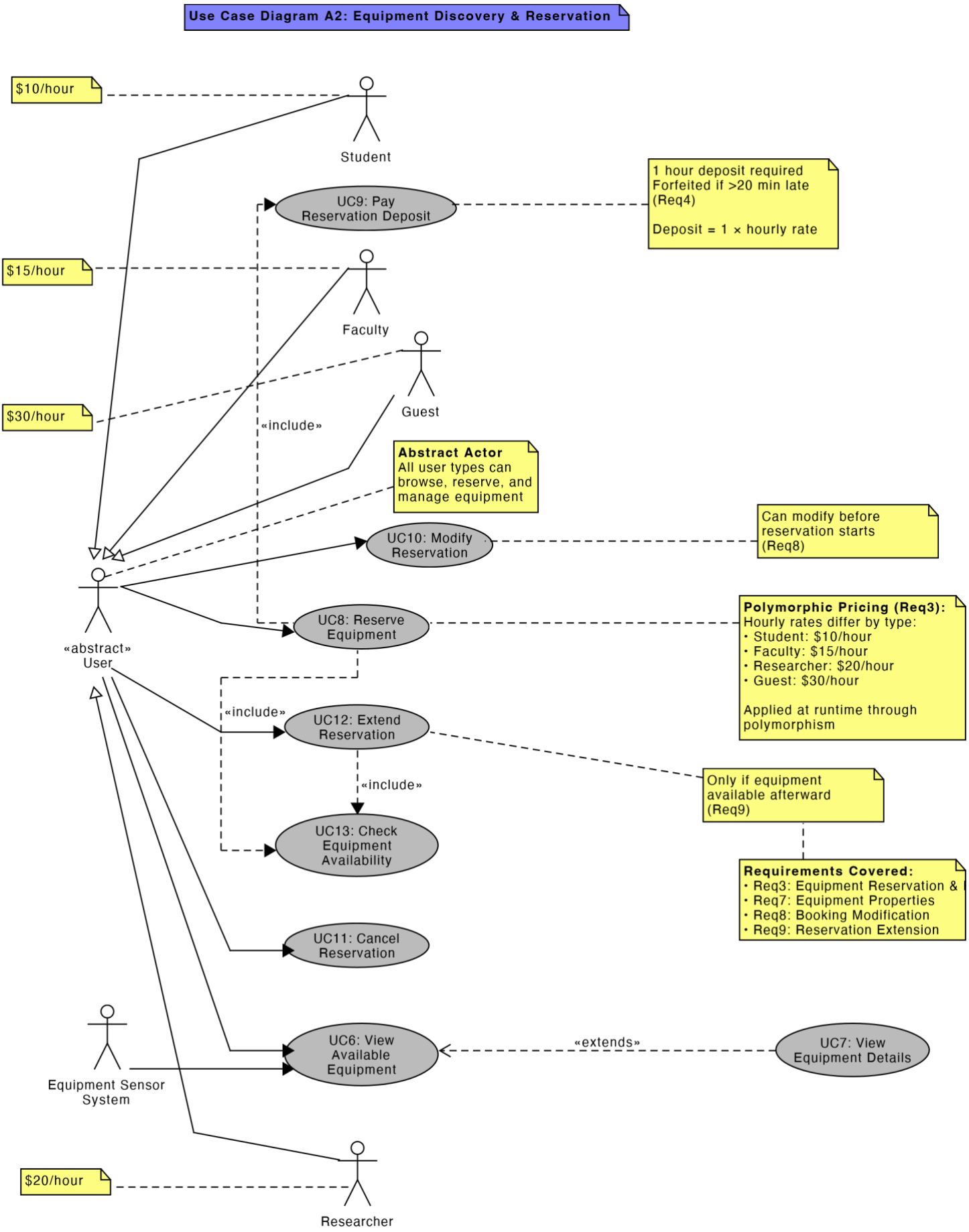
- Processes institutional accounts
- Processes research grant payments

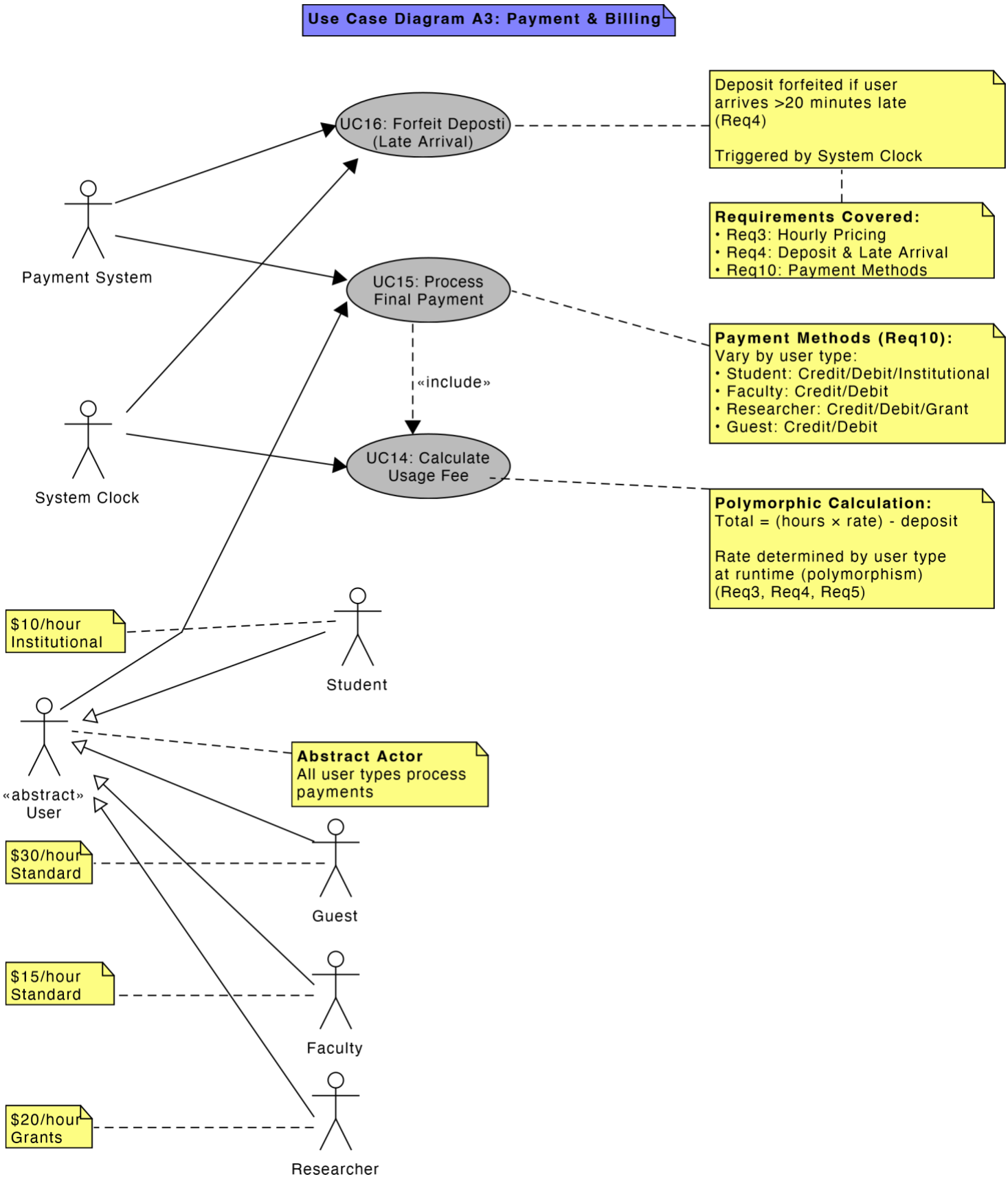
7. Department Approval System (Secondary Actor – External System) [Req1]

Interacts with:

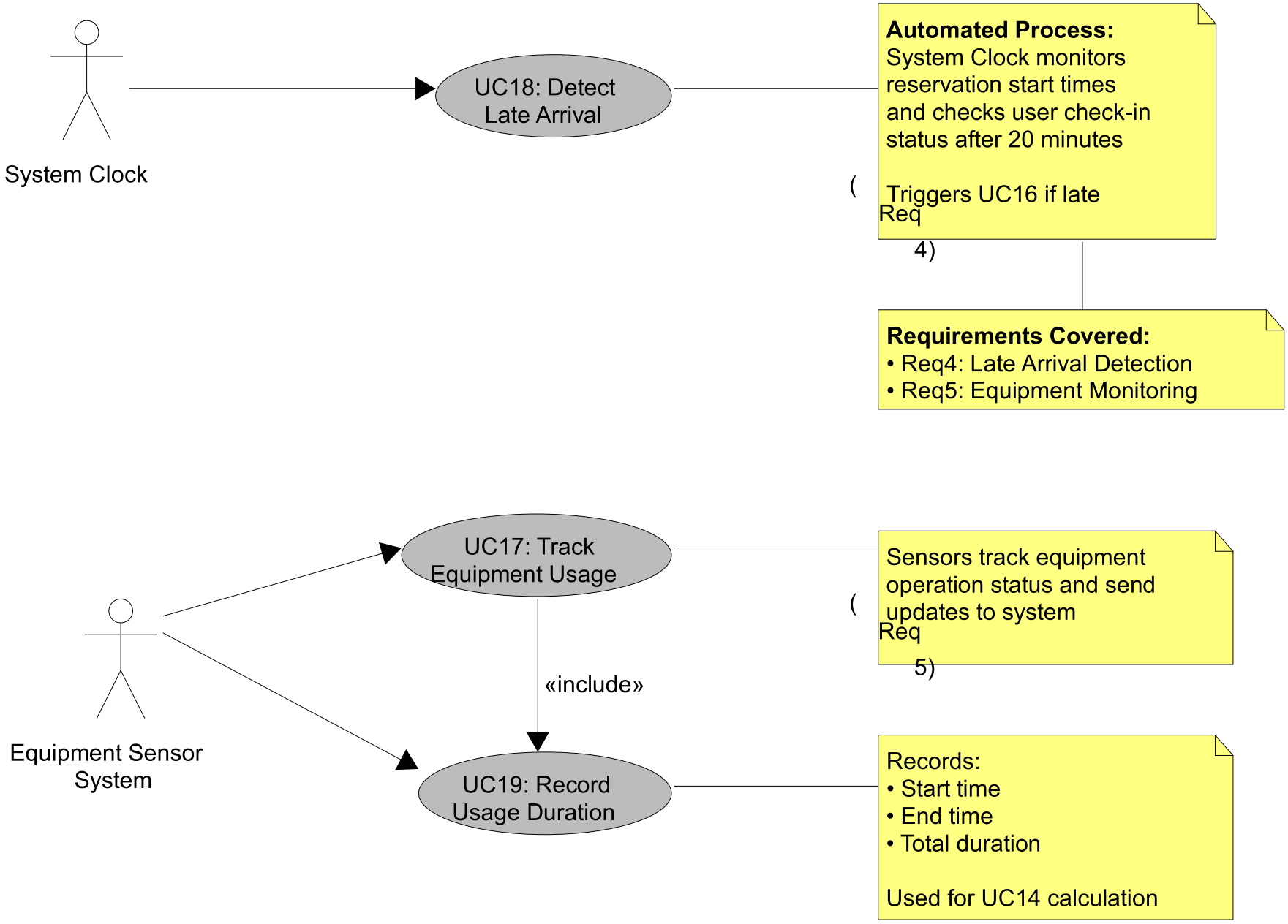
- University Account Approval during registration Notes / Constraints:
- Validates and approves university-affiliated accounts (Faculty, Researcher)

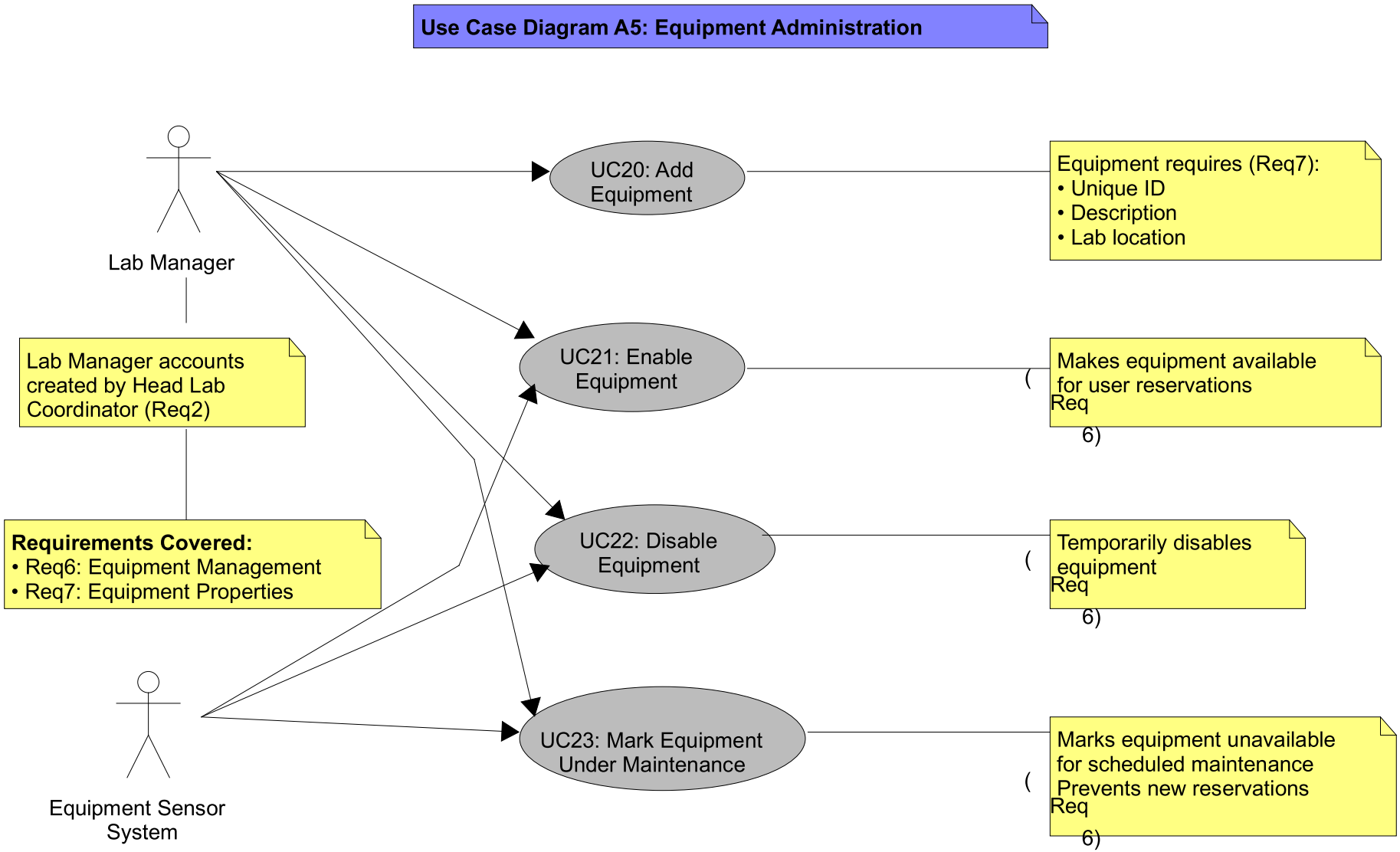

Use Case Diagram A1: Account Management
UC1: Register User Account
Requires unique email and strong password (uppercase, lowercase, numbers, symbols) (Req1)
«include»
UC3: Submit Identification/ Certification
«extend» (Faculty & Researcher)
«include»
ID type varies:
• Student: Student ID
• Faculty: Staff ID
• Researcher/Guest: Cert # (Req8)
«abstract» User
Requirements Covered:
• Req1: User Registration & Authentication
• Req8: User Identification
UC2: Validate Password Strength
UC4: Approve University-Affiliated Account
Required for Faculty and Researcher accounts only (Req1)
Student
$10/hour | Student ID | No approval
Faculty
$15/hour | Staff ID | Approval required
Researcher
$20/hour | Cert # | Approval required
Guest
$30/hour | Cert # | No approval
UC5: Login to System
Abstract Actor
All user types inherit registration and login capabilities
Department Approval System

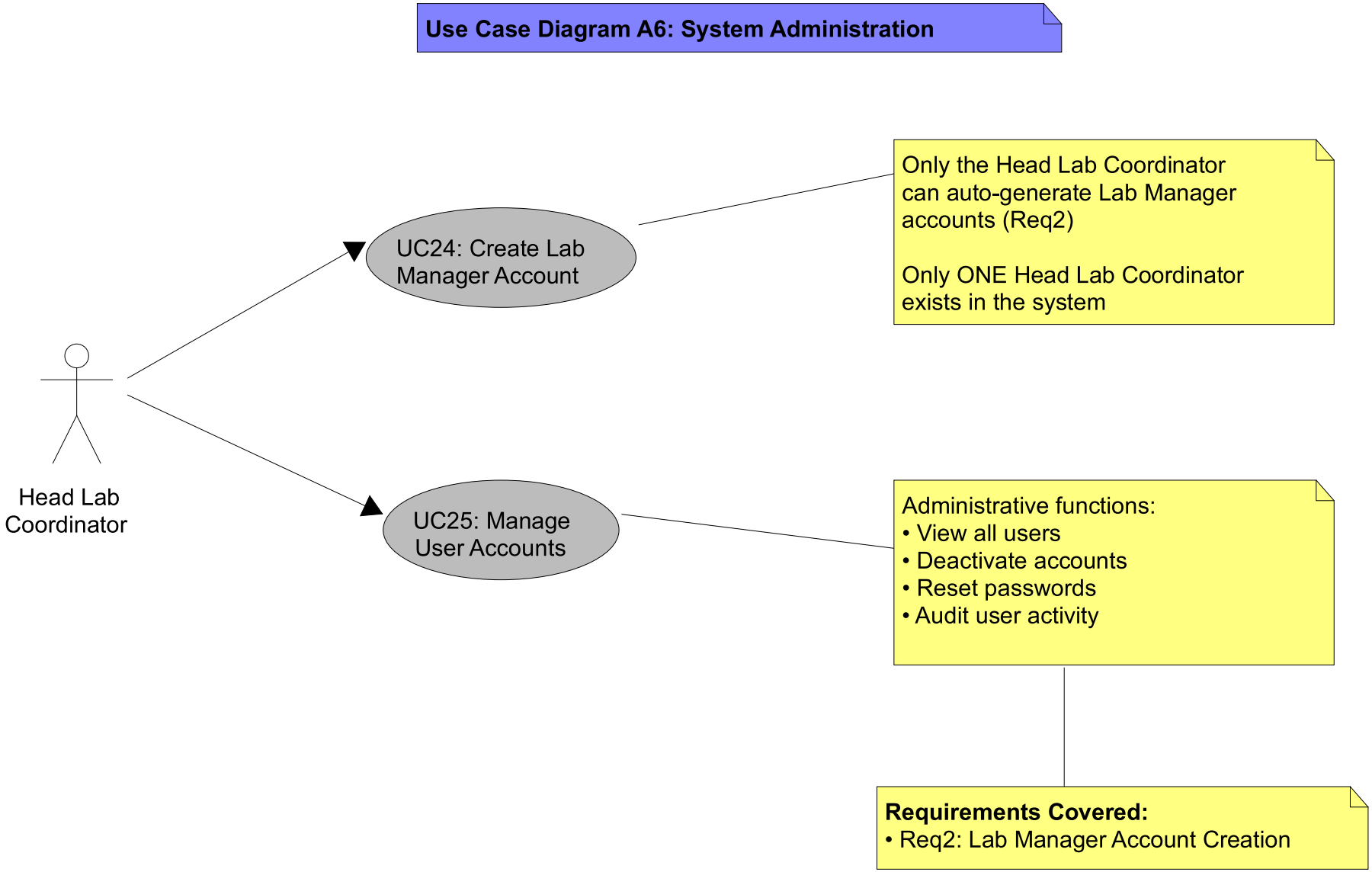

Use Case Diagram A2: Equipment Discovery & Reservation
$10/hour
Student
UC9: Pay Reservation Deposit
1 hour deposit required
Forfeited if >20 min late
(Req4)
Deposit = 1 × hourly rate
$15/hour
Faculty
Guest
$30/hour
«include»
Abstract Actor
All user types can browse, reserve, and manage equipment
UC10: Modify Reservation
Can modify before reservation starts
(Req8)
«abstract» User
UC8: Reserve Equipment
Polymorphic Pricing (Req3):
Hourly rates differ by type:
• Student: $10/hour
• Faculty: $15/hour
• Researcher: $20/hour
• Guest: $30/hour
Applied at runtime through polymorphism
«include»
UC12: Extend Reservation
«include»
Only if equipment available afterward
(Req9)
UC13: Check Equipment Availability
Requirements Covered:
• Req3: Equipment Reservation &
• Req7: Equipment Properties
• Req8: Booking Modification
• Req9: Reservation Extension
UC11: Cancel Reservation
Equipment Sensor System
UC6: View Available Equipment
«extends»
UC7: View Equipment Details
$20/hour
Researcher

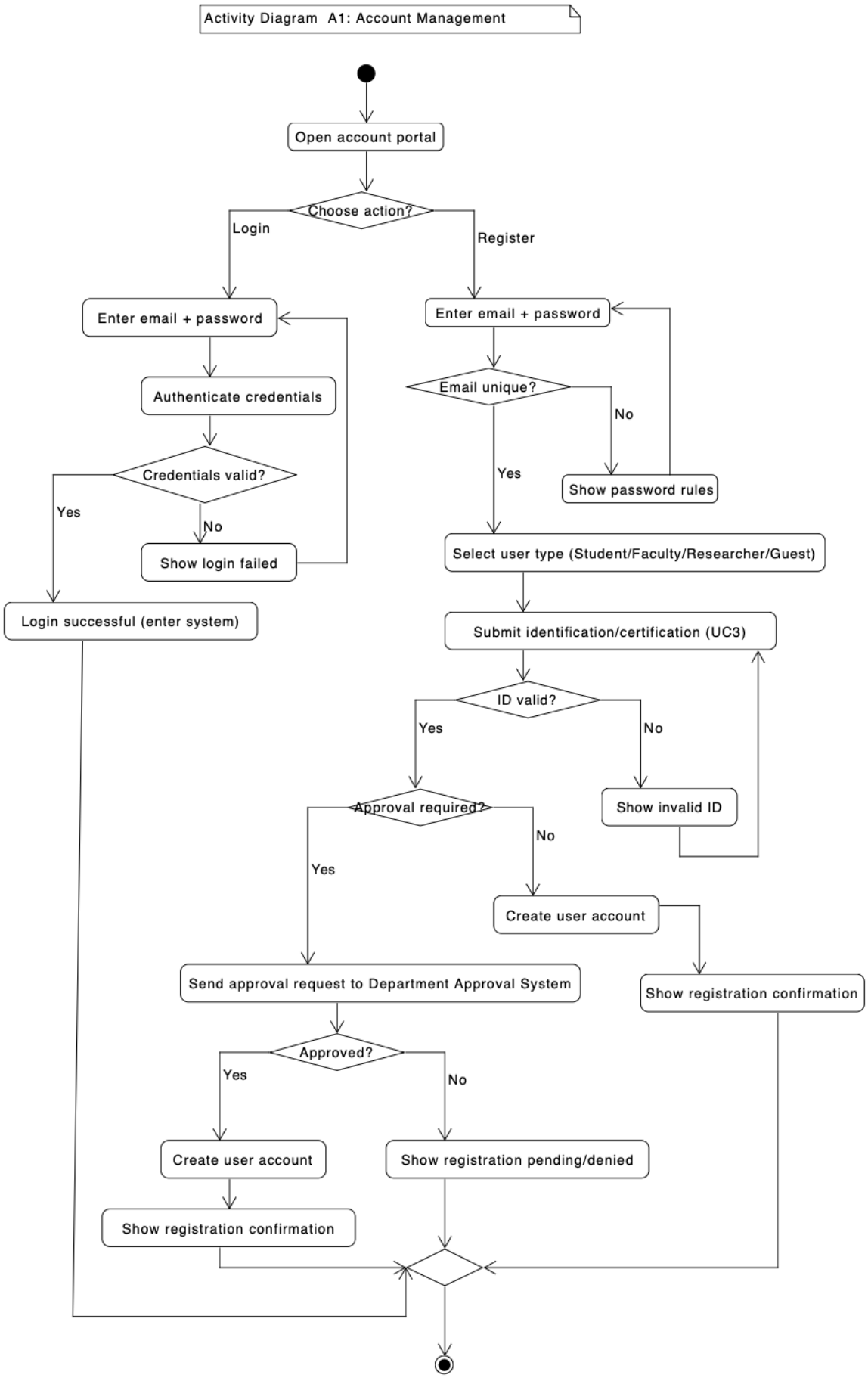

Use Case Diagram A3: Payment & Billing
UC16: Forfeit Deposti (Late Arrival)
Deposit forfeited if user arrives >20 minutes late (Req4)
Triggered by System Clock
Requirements Covered:
• Req3: Hourly Pricing
• Req4: Deposit & Late Arrival
• Req10: Payment Methods
Payment System
UC15: Process Final Payment
Payment Methods (Req10):
Vary by user type:
• Student: Credit/Debit/Institutional
• Faculty: Credit/Debit
• Researcher: Credit/Debit/Grant
• Guest: Credit/Debit
«include»
System Clock
UC14: Calculate Usage Fee
Polymorphic Calculation:
Total = (hours × rate) - deposit
Rate determined by user type at runtime (polymorphism) (Req3, Req4, Req5)
$10/hour Institutional
Student
«abstract» User
Abstract Actor
All user types process payments
$30/hour Standard
Guest
$15/hour Standard
Faculty
$20/hour Grants
Researcher

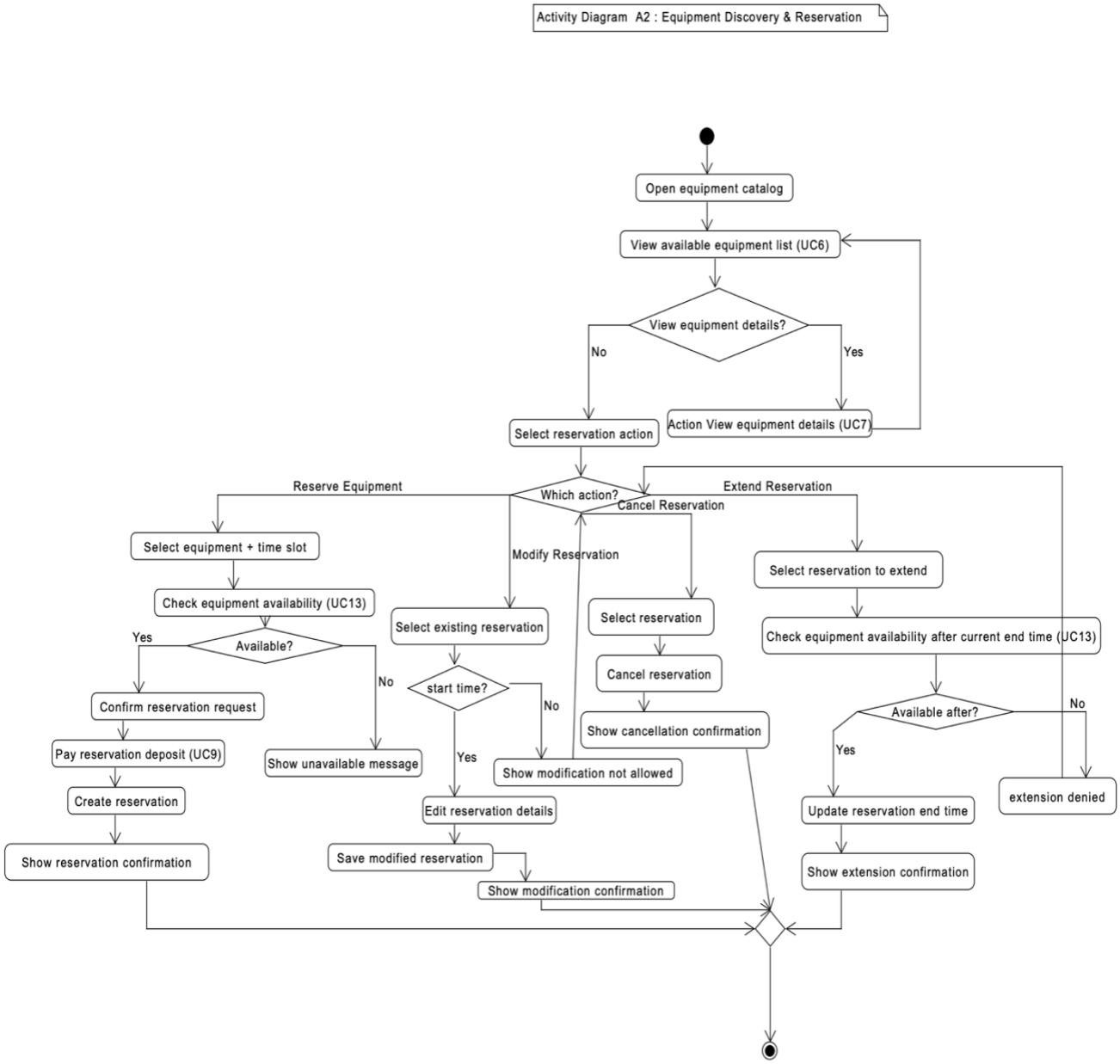
Use Case Diagram A4: Monitoring & Enforcement



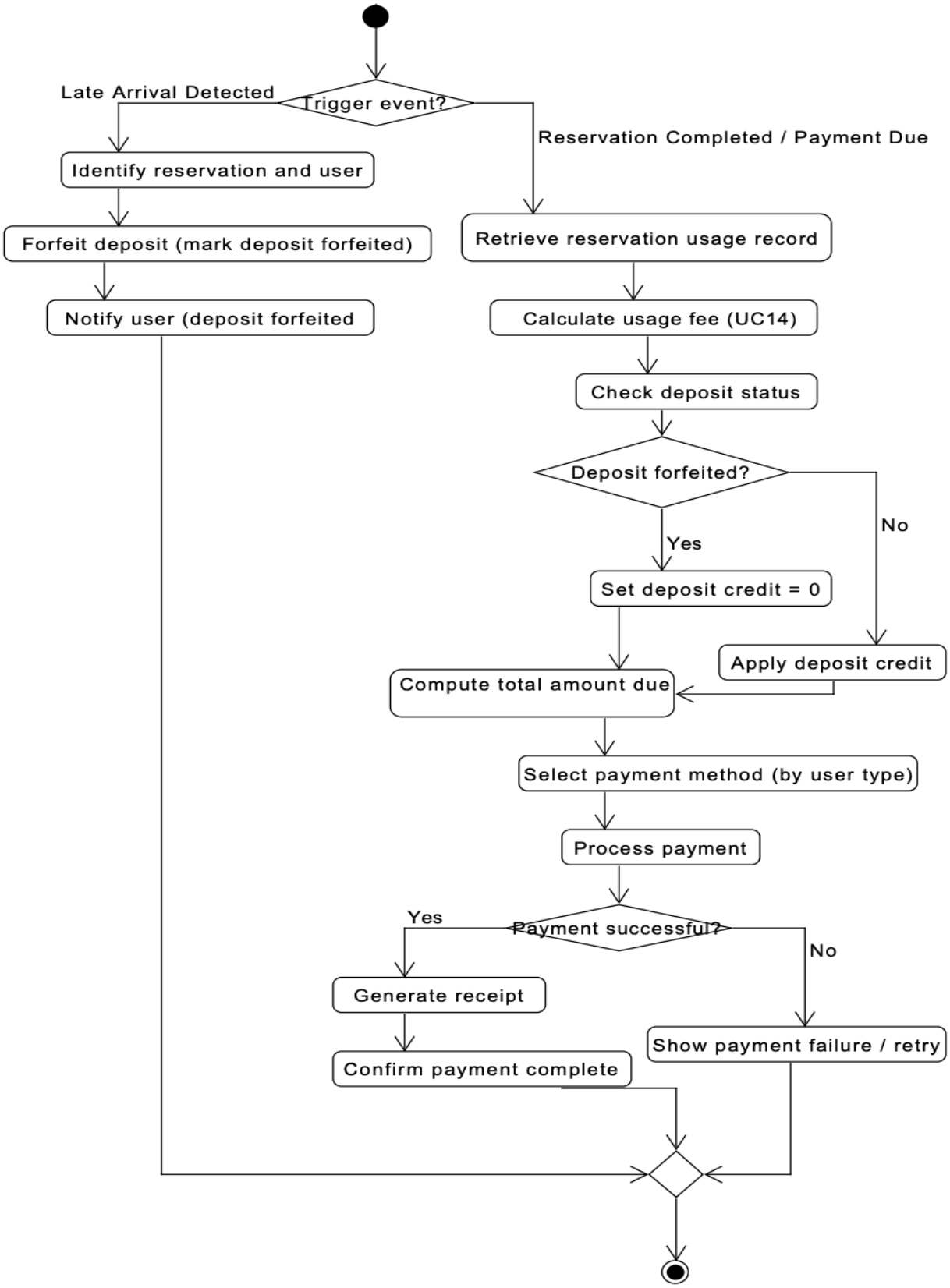

Use Case Diagram A5: Equipment Administration
Lab Manager
Equipment Sensor System
UC20: Add Equipment
UC21: Enable Equipment
UC22: Disable Equipment
UC23: Mark Equipment Under Maintenance
Equipment requires (Req7):
• Unique ID
• Description
• Lab location
Makes equipment available for user reservations
Temporarily disables equipment
Marks equipment unavailable for scheduled maintenance
Prevents new reservations
Lab Manager accounts created by Head Lab Coordinator (Req2)
Requirements Covered:
• Req6: Equipment Management
• Req7: Equipment Properties

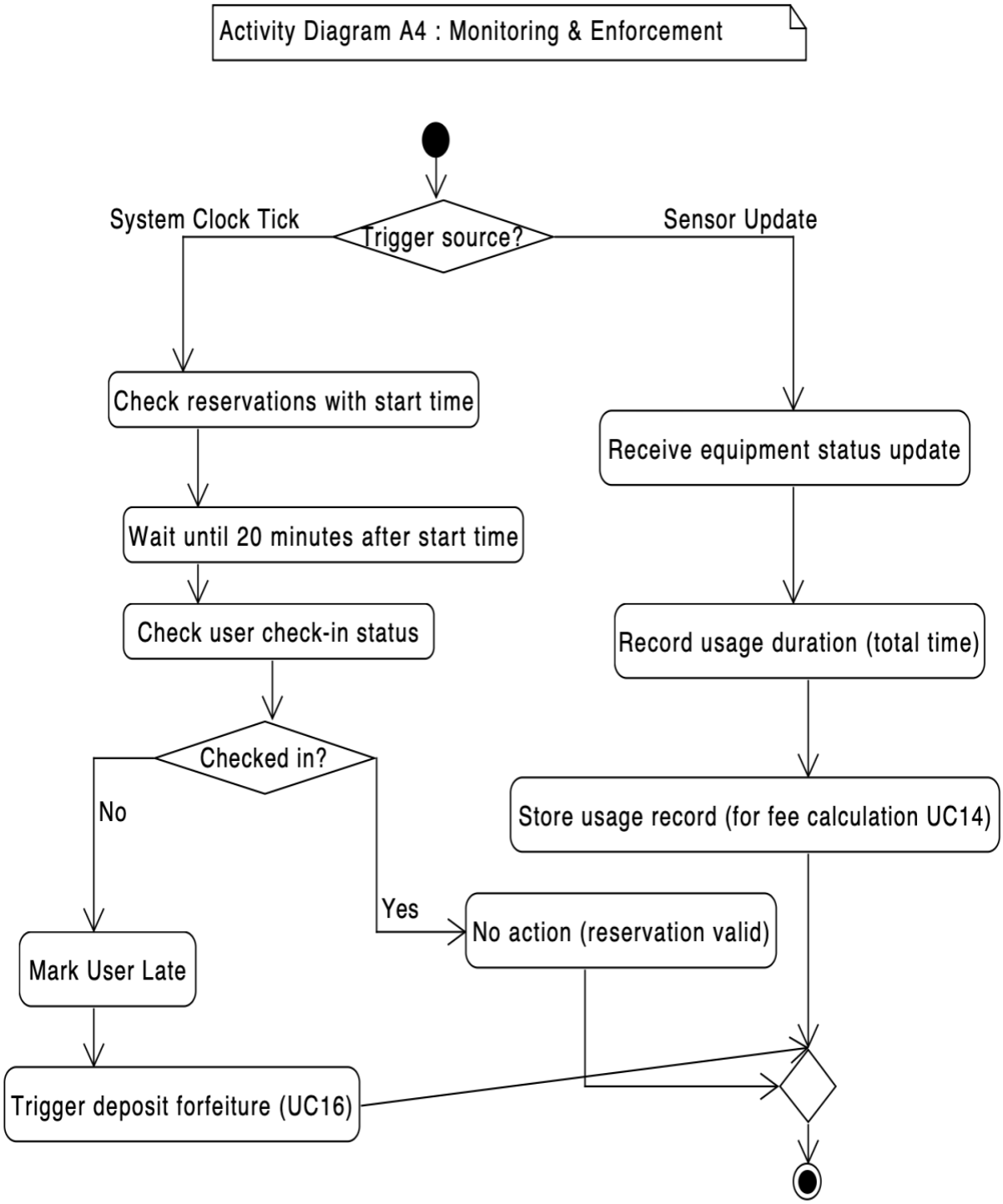

Use Case Diagram A6: System Administration
Head Lab Coordinator
UC24: Create Lab Manager Account
UC25: Manage User Accounts
Only the Head Lab Coordinator can auto-generate Lab Manager accounts (Req2)
Only ONE Head Lab Coordinator exists in the system
Administrative functions:
• View all users
• Deactivate accounts
• Reset passwords
• Audit user activity
Requirements Covered:
• Req2: Lab Manager Account Creation


Activity Diagram A1: Account Management
Open account portal
Choose action?
Login
Register
Enter email + password
Authenticate credentials
Credentials valid?
Yes
No
Show login failed
Login successful (enter system)
Enter email + password
Email unique?
No
Show password rules
Yes
Select user type (Student/Faculty/Researcher/Guest)
Submit identification/certification (UC3)
ID valid?
Yes
No
Show invalid ID
Approval required?
No
Yes
Create user account
Show registration confirmation
Send approval request to Department Approval System
Approved?
Yes
No
Create user account
Show registration pending/denied
Show registration confirmation

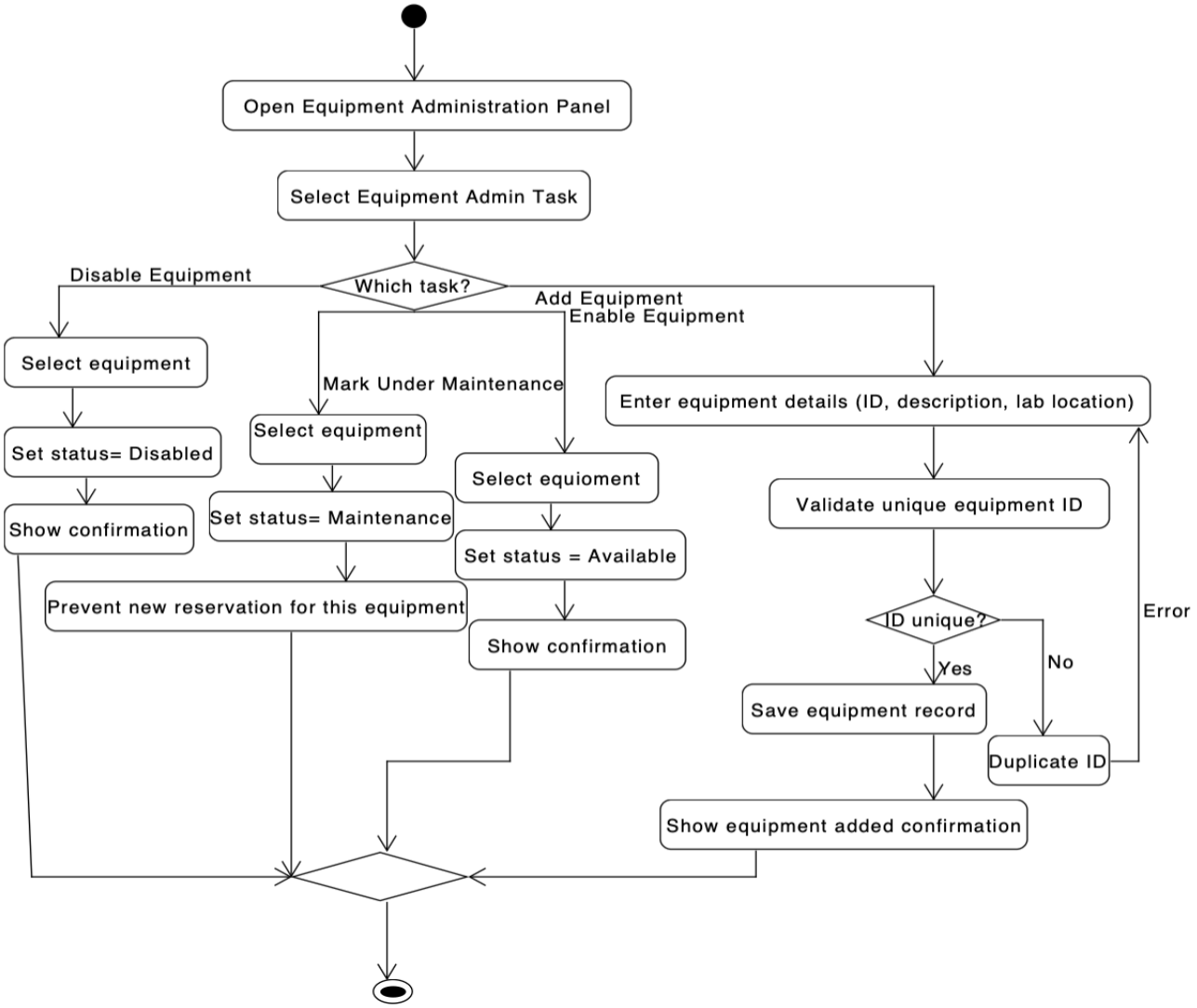

Activity Diagram A2 : Equipment Discovery & Reservation
Open equipment catalog
View available equipment list (UC6)
View equipment details?
No
Yes
Action View equipment details (UC7)
Select reservation action
Which action?
Reserve Equipment
Extend Reservation
Cancel Reservation
Modify Reservation
Select equipment + time slot
Check equipment availability (UC13)
Available?
Yes
No
Confirm reservation request
Pay reservation deposit (UC9)
Create reservation
Show reservation confirmation
Show unavailable message
Select existing reservation
start time?
No
Yes
Edit reservation details
Save modified reservation
Show modification confirmation
Show modification not allowed
Select reservation
Cancel reservation
Show cancellation confirmation
Select reservation to extend
Check equipment availability after current end time (UC13)
Available after?
Yes
No
Update reservation end time
Show extension confirmation
extension denied

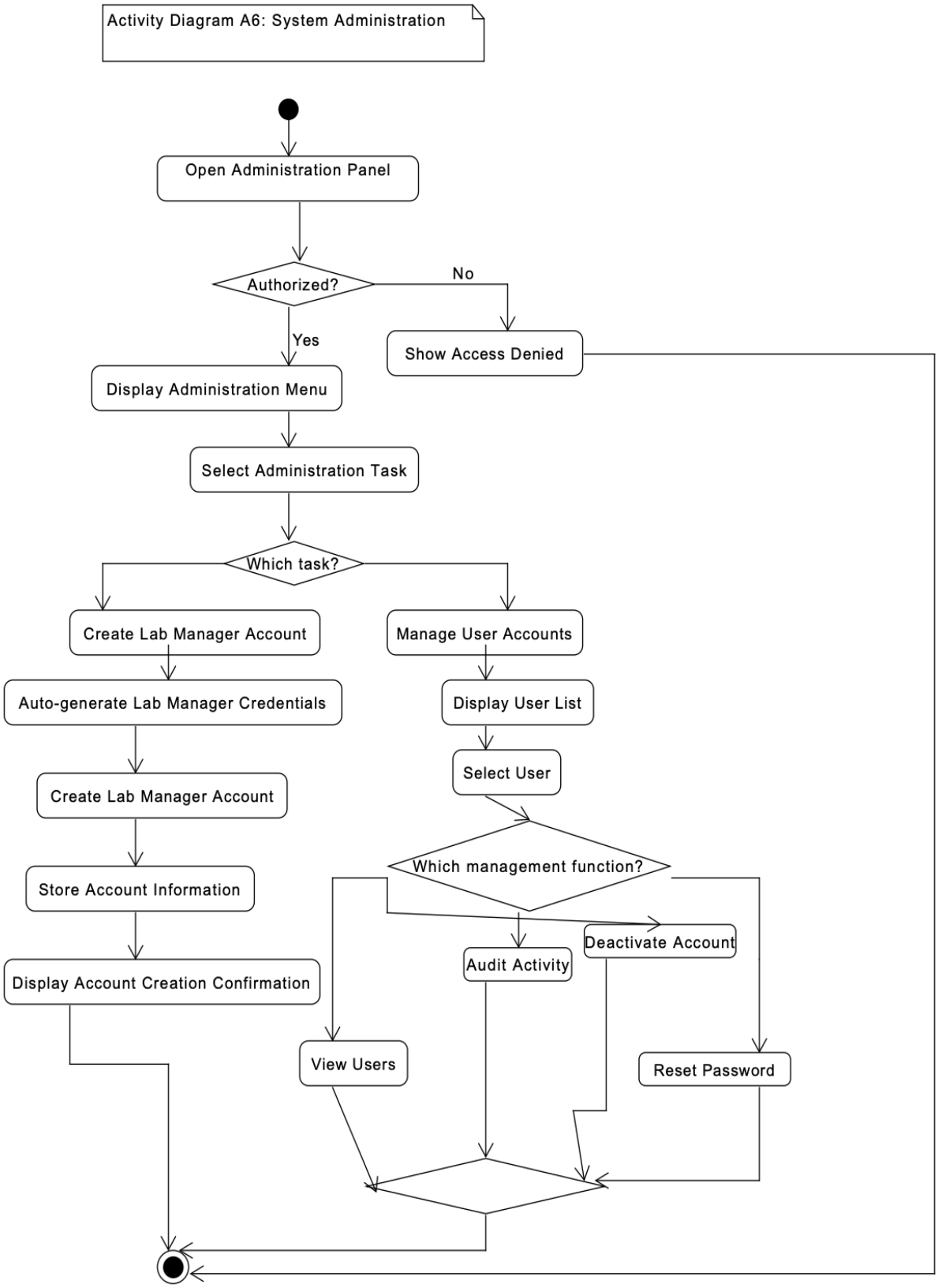
Activity Diagram A3 : Payment & Billing



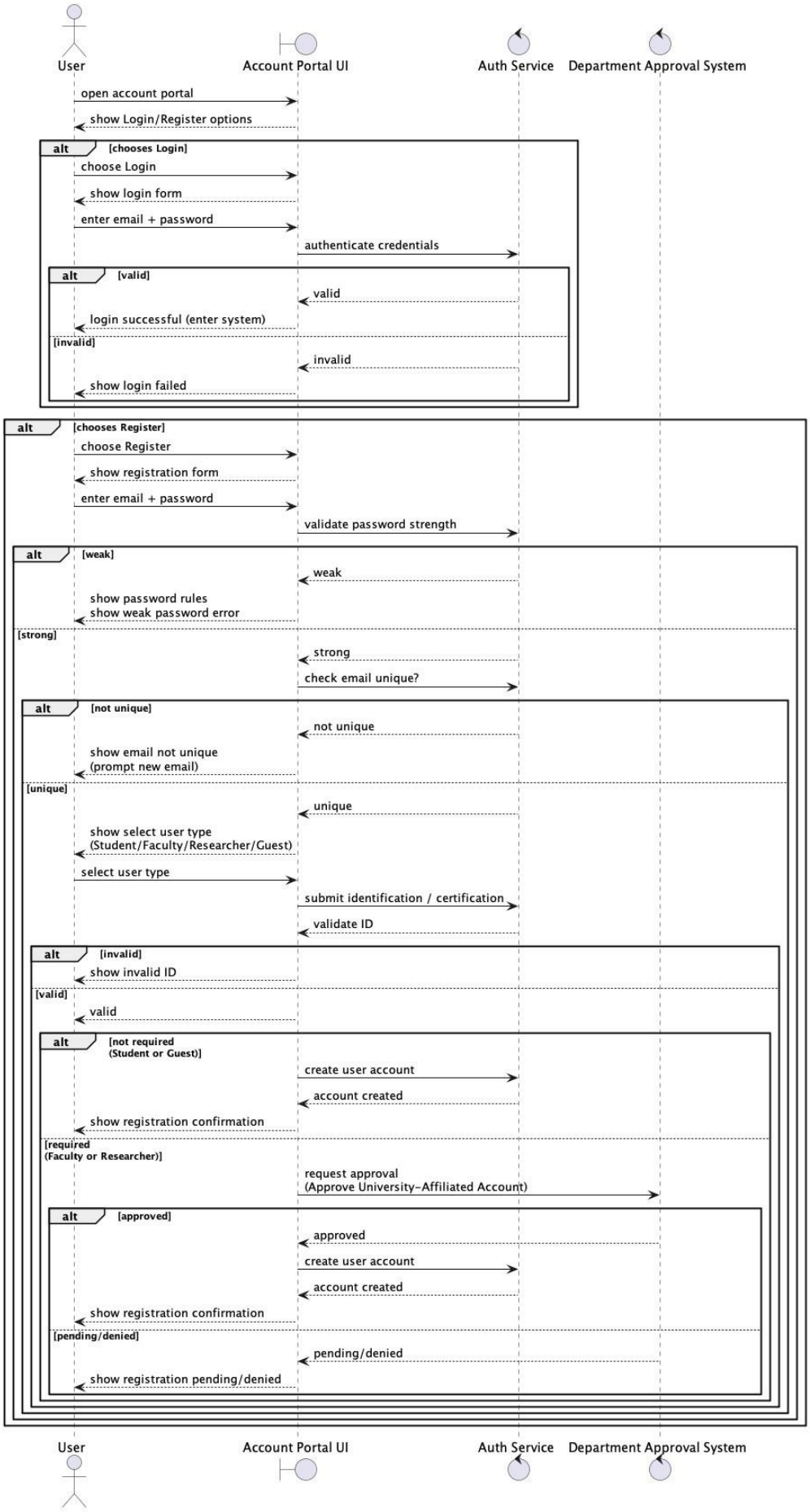

Activity Diagram A4 : Monitoring & Enforcement
Trigger source?
System Clock Tick
Sensor Update
Check reservations with start time
Wait until 20 minutes after start time
Check user check-in status
Checked in?
No
Yes
Mark User Late
Trigger deposit forfeiture (UC16)
No action (reservation valid)
Receive equipment status update
Record usage duration (total time)
Store usage record (for fee calculation UC14)

Activity Diagram A5 : Equipment Administration

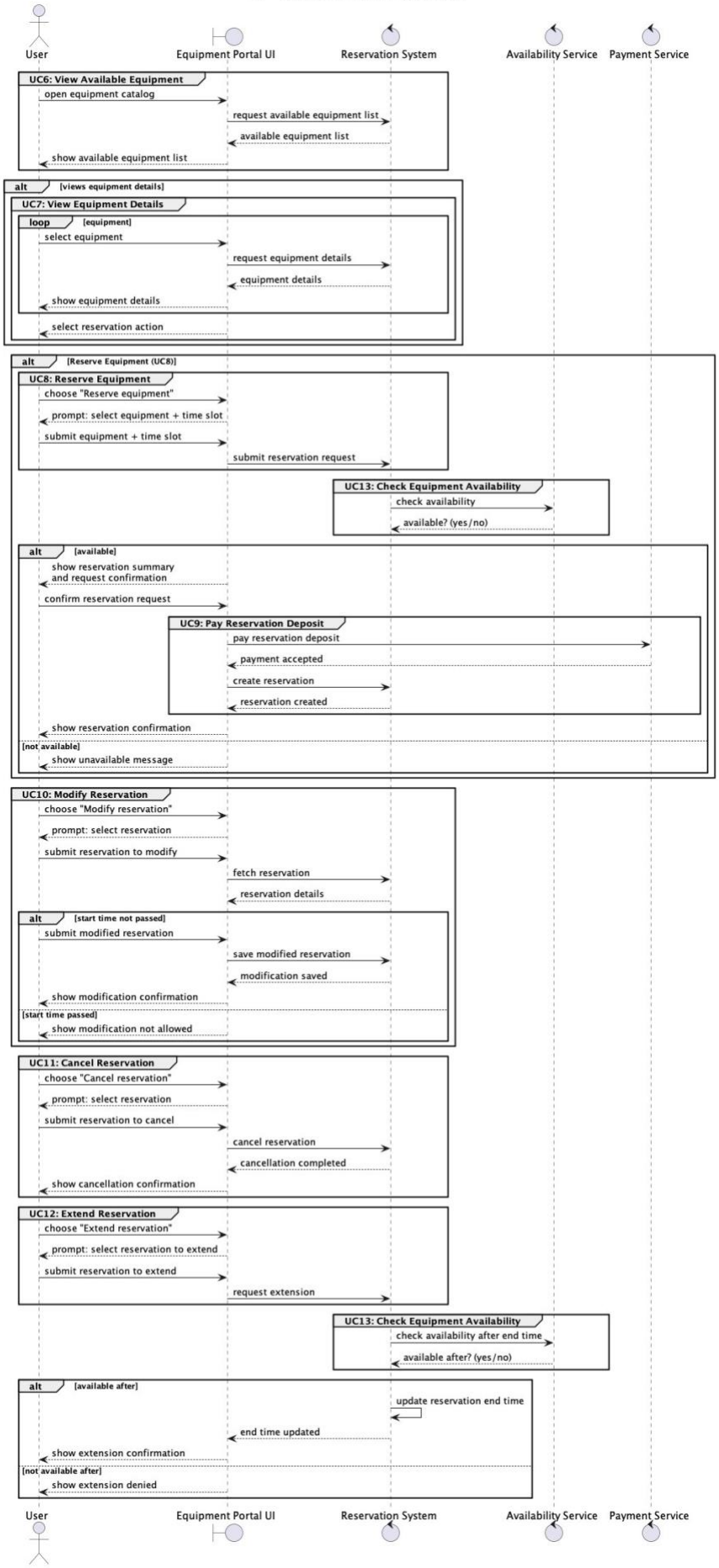


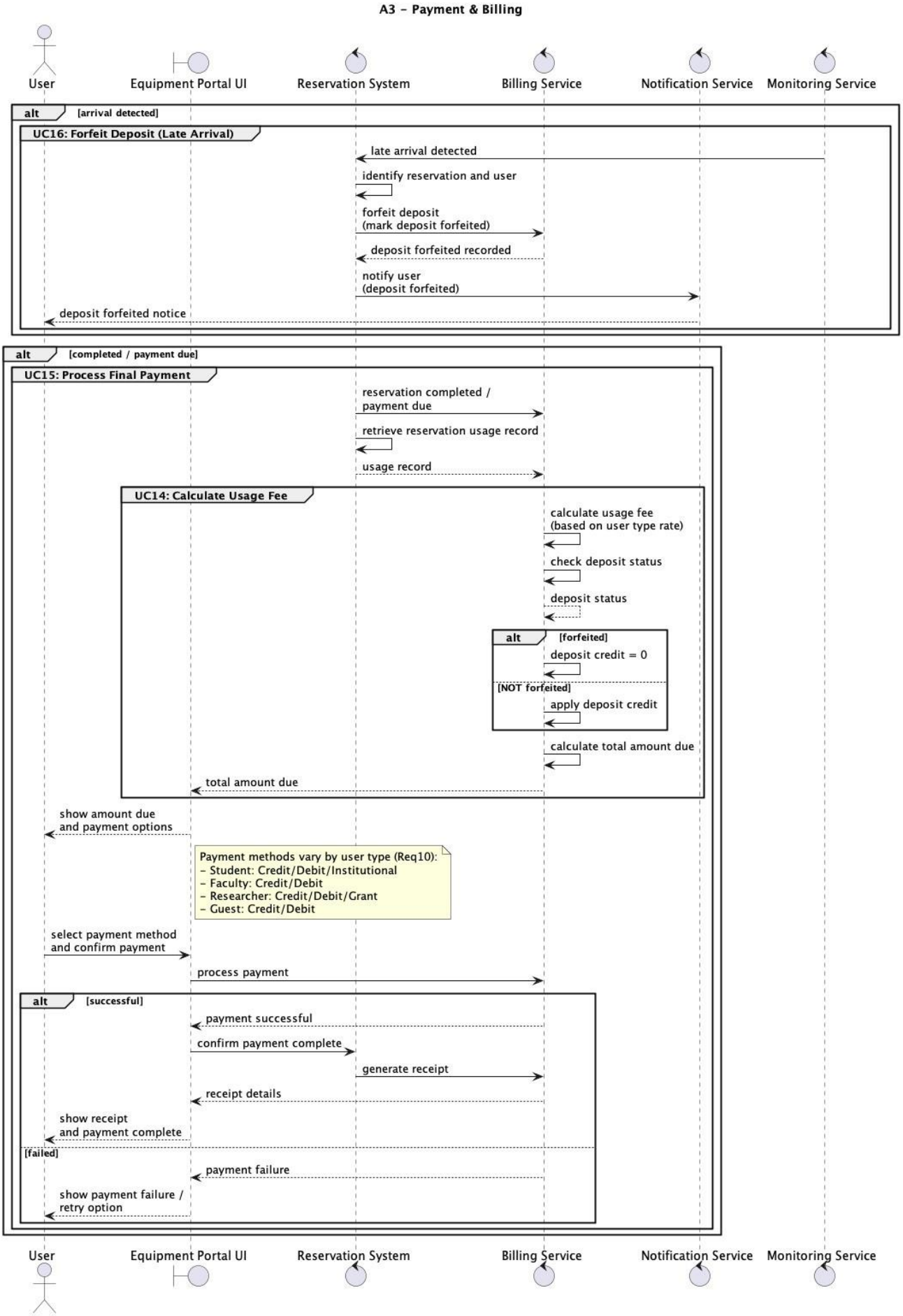

Activity Diagram A6: System Administration
Open Administration Panel
Authorized?
No
Show Access Denied
Yes
Display Administration Menu
Select Administration Task
Which task?
Create Lab Manager Account
Manage User Accounts
Auto-generate Lab Manager Credentials
Display User List
Select User
Create Lab Manager Account
Which management function?
Store Account Information
Deactivate Account
Audit Activity
Display Account Creation Confirmation
View Users
Reset Password

A1 – Account Management: Login & Registration

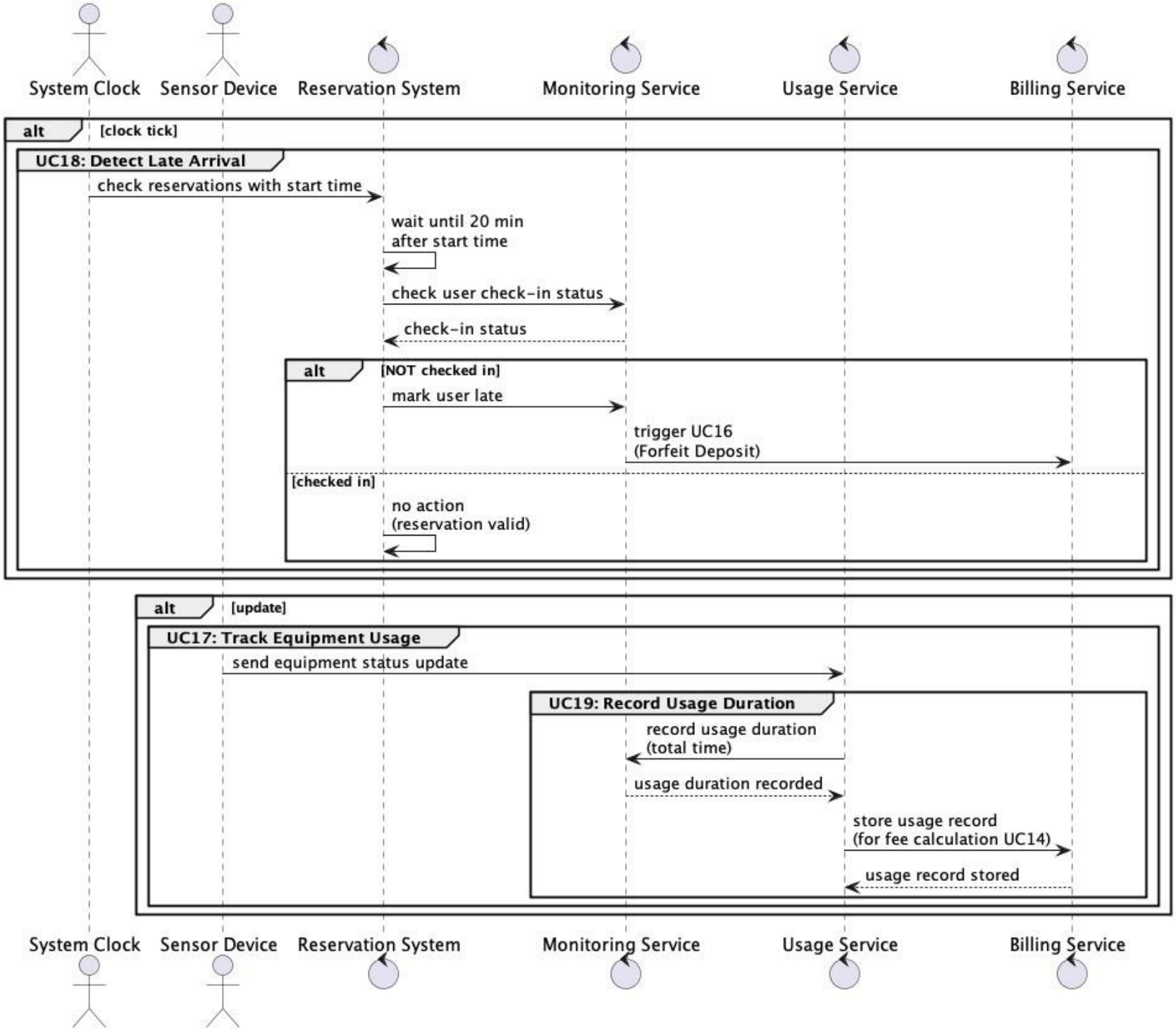


A2 – Equipment Discovery & Reservation

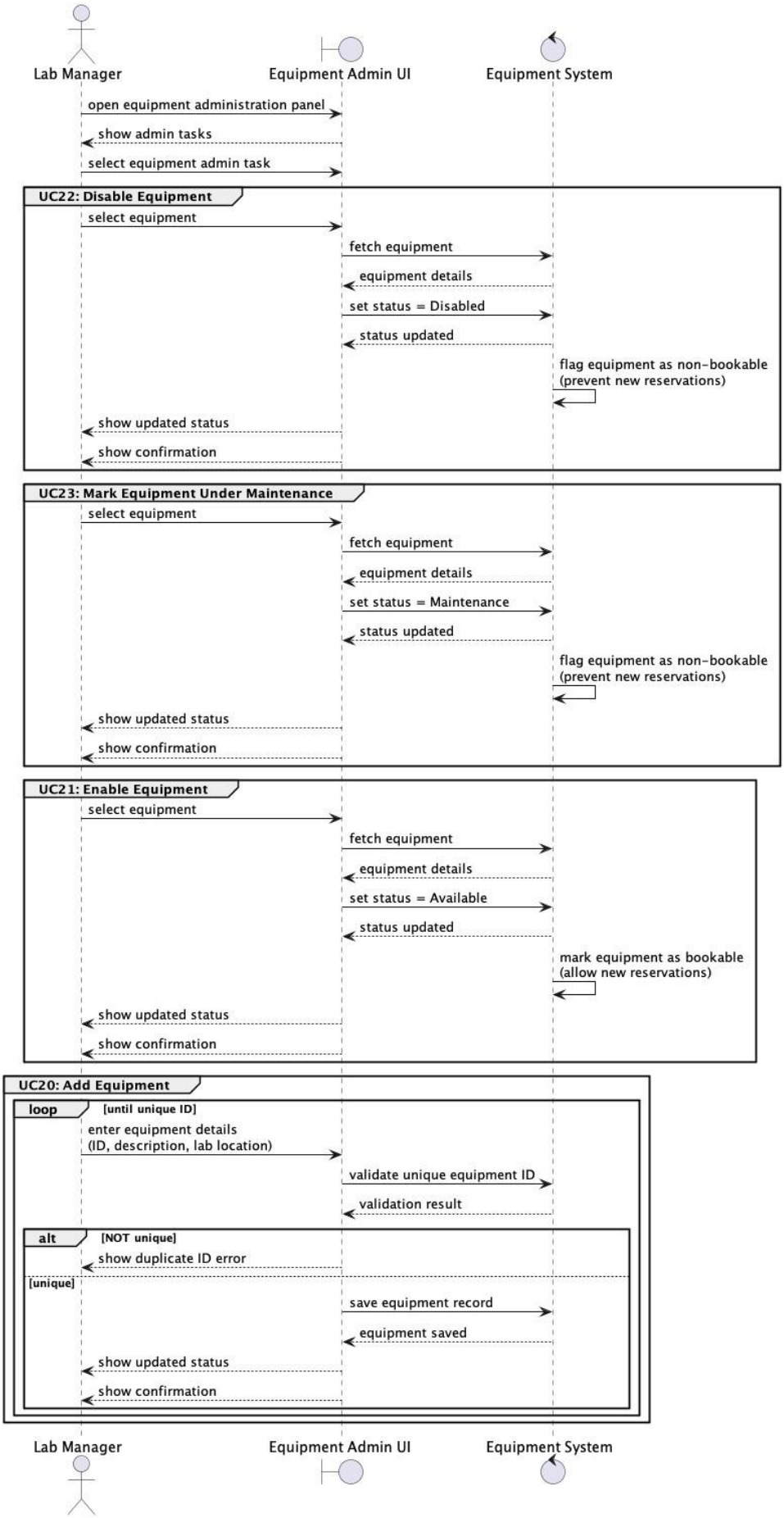



A3 – Payment & Billing
User
Equipment Portal UI
Reservation System
Billing Service
Notification Service
Monitoring Service
alt [arrival detected]
UC16: Forfeit Deposit (Late Arrival)
late arrival detected
identify reservation and user
forfeit deposit
(mark deposit forfeited)
deposit forfeited recorded
notify user
(deposit forfeited)
deposit forfeited notice
alt [completed / payment due]
UC15: Process Final Payment
reservation completed /
payment due
retrieve reservation usage record
usage record
UC14: Calculate Usage Fee
calculate usage fee
(based on user type rate)
check deposit status
deposit status
alt [forfeited]
deposit credit = 0
[NOT forfeited]
apply deposit credit
calculate total amount due
total amount due
show amount due
and payment options
Payment methods vary by user type (Req10):
- Student: Credit/Debit/Institutional
- Faculty: Credit/Debit
- Researcher: Credit/Debit/Grant
- Guest: Credit/Debit
select payment method
and confirm payment
process payment
alt [successful]
payment successful
confirm payment complete
generate receipt
receipt details
show receipt
and payment complete
[failed]
payment failure
show payment failure /
retry option
User
Equipment Portal UI
Reservation System
Billing Service
Notification Service
Monitoring Service

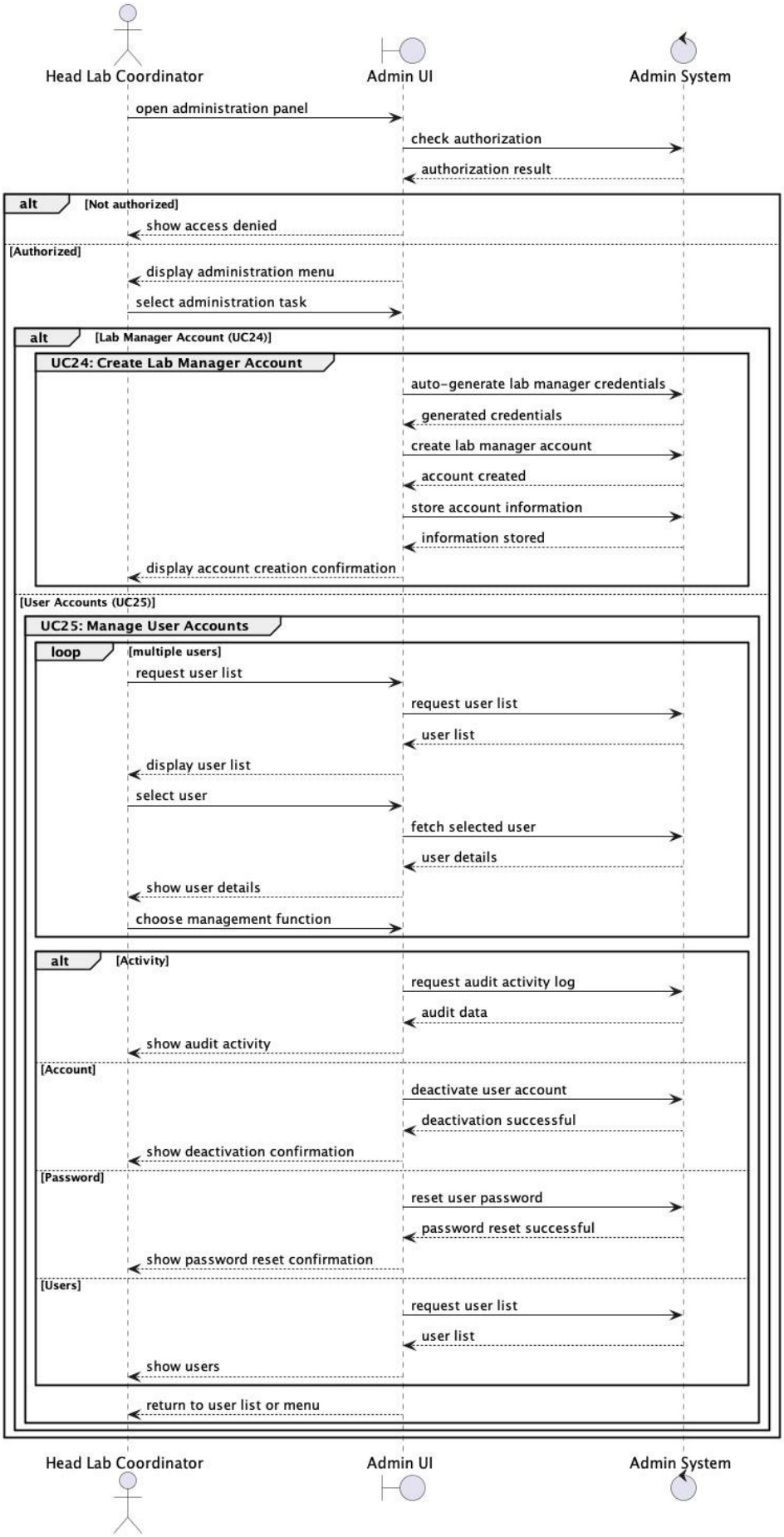
A4 – Monitoring & Enforcement



A5 – Equipment Administration



A6 – System Administration



Part II: Design

1. **User** (abstract)
   Attributes:
   - userId: String
   - name: String
   - email: String
   - passwordHash: String
   - idNumber: String
   - accountStatus: AccountStatus Methods:
   - register(email: String, password: String, idNumber: String): void
   - login(email: String, password: String): boolean
   - getHourlyRate(): double Generalization:
   - Student extends User
   - Faculty extends User
   - Researcher extends User
   - Guest extends User

2. **Student / Faculty /**
   Researcher / Guest
   Methods:
   - getHourlyRate(): double

3. **LabManager**
   Attributes:
   - managerId: String
   - name: String
   - email: String Methods:
   - addEquipment(equipmentId: String, description: String, labLocation: String): Equipment
   - enableEquipment(e: Equipment): void
   - disableEquipment(e: Equipment): void
   - markUnderMaintenance(e: Equipment): void

4. **HeadLabCoordinator**
   Attributes:
   - coordinatorId: String
   - name: String
   - email: String Methods:
   - createLabManagerAccount(name: String, email: String): LabManager
5. **Equipment**
   Attributes:

- equipmentId: String
- description: String
- labLocation: String
- status: EquipmentStatus Methods:
- enable(): void
- disable(): void
- markUnderMaintenance(): void
- isAvailable(start: LocalDateTime, end: LocalDateTime): boolean

6. **Reservation**
   Attributes:
   - reservationId: String
   - startT ime: LocalDateTime
   - endTime: LocalDateTime
   - status: ReservationStatus
   - depositAmount: double
   - depositForfeited: boolean Methods:
   - modify(newStart: LocalDateTime, newEnd: LocalDateTime): void
   - cancel(): void
   - extend(newEnd: LocalDateTime): void
   - durationHours(): double

7. **Payment**
   Attributes:
   - paymentId: String
   - amount: double
   - method: PaymentMethod
   - status: PaymentStatus
   - timestamp: LocalDateTime
   - type: PaymentType Methods:
   - process(gateway: PaymentGateway): boolean
   - refund(gateway: PaymentGateway): boolean

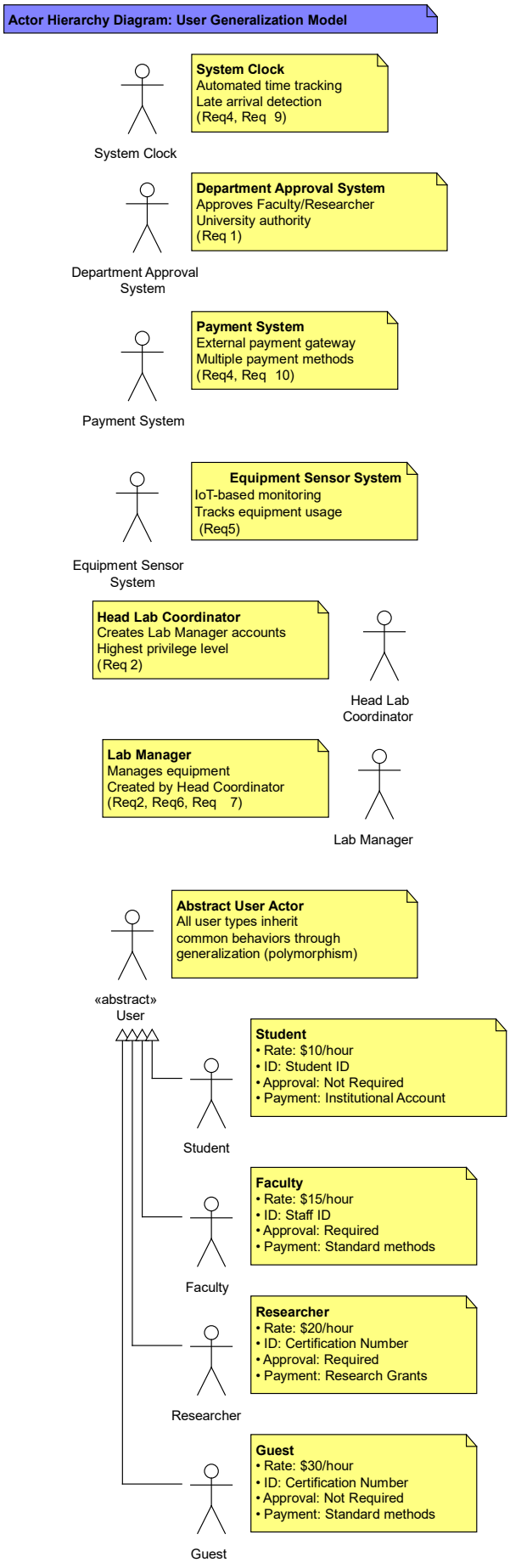
8. **UsageRecord**
   Attributes:
   - recordId: String
   - startTime: LocalDateTime
   - endTime: LocalDateTime
   - durationMinutes: int
   - operationStatus: String Methods:
   - computeDurationMinutes(): int

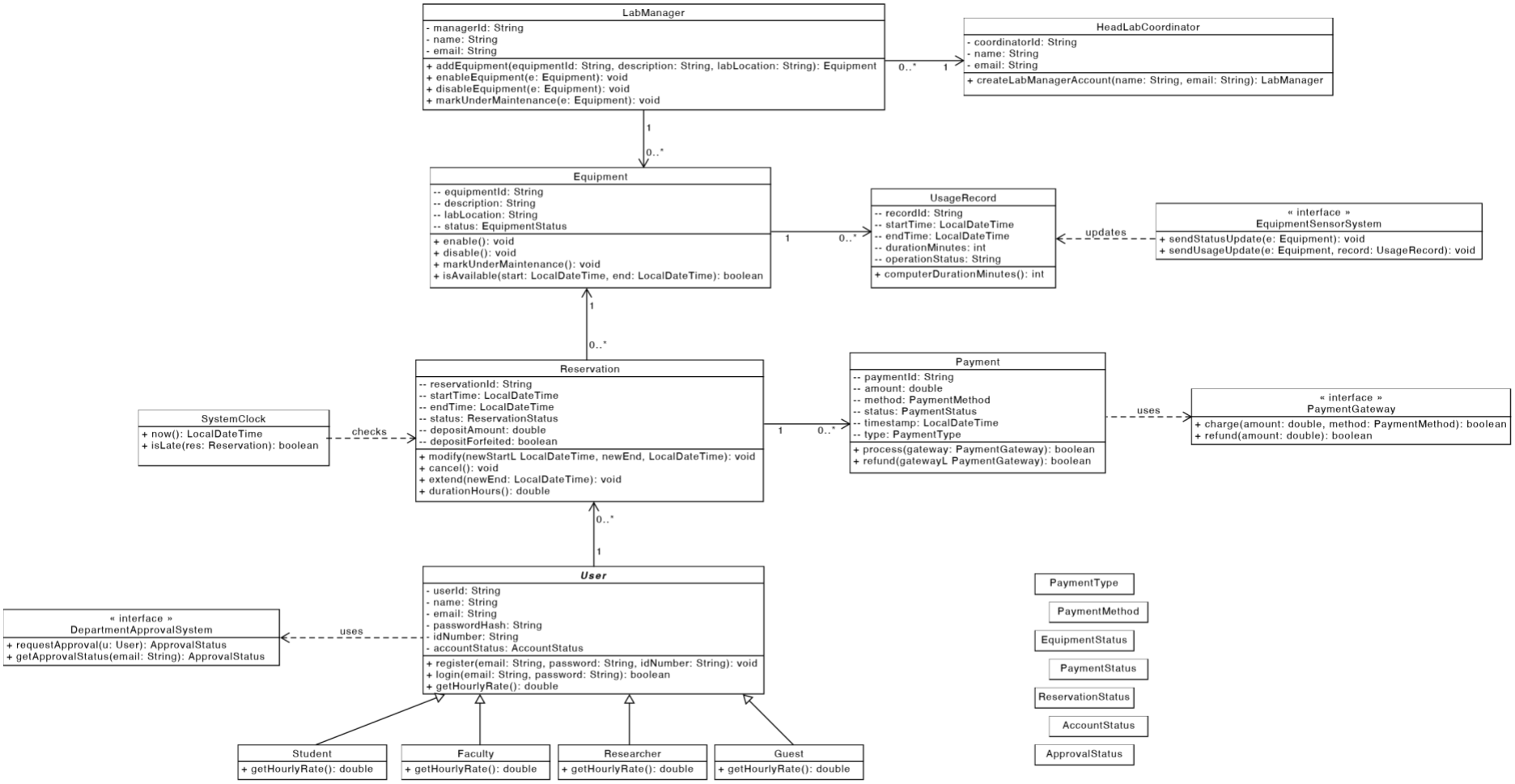
9. **PaymentGateway**
   Methods:
   - charge(amount: double, method: PaymentMethod): boolean
   - refund(amount: double): boolean

10. **DepartmentApproval System** Methods:
    - requestApproval(u: User): ApprovalStatus
    - getApprovalStatus(email: String): ApprovalStatus

11. **SystemClock**
    Methods:
    - now(): LocalDateTime
    - isLate(res: Reservation): boolean

12. **EquipmentSensorSystem** Methods:
    - sendStatusUpdate(e: Equipment): void
    - sendUsageUpdate(e: Equipment, record: UsageRecord): void


Actor Hierarchy Diagram: User Generalization Model
System Clock
System Clock
Automated time tracking
Late arrival detection
(Req4, Req 9)
Department Approval System
Department Approval System
Approves Faculty/Researcher
University authority
(Req 1)
Payment System
Payment System
External payment gateway
Multiple payment methods
(Req4, Req 10)
Equipment Sensor System
Equipment Sensor System
IoT-based monitoring
Tracks equipment usage
(Req5)
Head Lab Coordinator
Head Lab Coordinator
Creates Lab Manager accounts
Highest privilege level
(Req 2)
Lab Manager
Lab Manager
Manages equipment
Created by Head Coordinator
(Req2, Req6, Req 7)
«abstract» User
Abstract User Actor
All user types inherit
common behaviors through
generalization (polymorphism)
Student
Student
• Rate: $10/hour
• ID: Student ID
• Approval: Not Required
• Payment: Institutional Account
Faculty
Faculty
• Rate: $15/hour
• ID: Staff ID
• Approval: Required
• Payment: Standard methods
Researcher
Researcher
• Rate: $20/hour
• ID: Certification Number
• Approval: Required
• Payment: Research Grants
Guest
Guest
• Rate: $30/hour
• ID: Certification Number
• Approval: Not Required
• Payment: Standard methods


LabManager
- managerId: String
- name: String
- email: String
+ addEquipment(equipmentId: String, description: String, labLocation: String): Equipment
+ enableEquipment(e: Equipment): void
+ disableEquipment(e: Equipment): void
+ markUnderMaintenance(e: Equipment): void
0..*
1
HeadLabCoordinator
- coordinatorId: String
- name: String
- email: String
+ createLabManagerAccount(name: String, email: String): LabManager
1
0..*
Equipment
-- equipmentId: String
-- description: String
-- labLocation: String
-- status: EquipmentStatus
+ enable(): void
+ disable(): void
+ markUnderMaintenance(): void
+ isAvailable(start: LocalDateTime, end: LocalDateTime): boolean
1
0..*
UsageRecord
-- recordId: String
-- startTime: LocalDateTime
-- endTime: LocalDateTime
-- durationMinutes: int
-- operationStatus: String
+ computerDurationMinutes(): int
updates
« interface »
EquipmentSensorSystem
+ sendStatusUpdate(e: Equipment): void
+ sendUsageUpdate(e: Equipment, record: UsageRecord): void
1
0..*
Reservation
-- reservationId: String
-- startTime: LocalDateTime
-- endTime: LocalDateTime
-- status: ReservationStatus
-- depositAmount: double
-- depositForfeited: boolean
+ modify(newStartL LocalDateTime, newEnd, LocalDateTime): void
+ cancel(): void
+ extend(newEnd: LocalDateTime): void
+ durationHours(): double
SystemClock
+ now(): LocalDateTime
+ isLate(res: Reservation): boolean
checks
1
0..*
Payment
-- paymentId: String
-- amount: double
-- method: PaymentMethod
-- status: PaymentStatus
-- timestamp: LocalDateTime
-- type: PaymentType
+ process(gateway: PaymentGateway): boolean
+ refund(gatewayL PaymentGateway): boolean
uses
« interface »
PaymentGateway
+ charge(amount: double, method: PaymentMethod): boolean
+ refund(amount: double): boolean
0..*
1
User
- userId: String
- name: String
- email: String
- passwordHash: String
- idNumber: String
- accountStatus: AccountStatus
+ register(email: String, password: String, idNumber: String): void
+ login(email: String, password: String): boolean
+ getHourlyRate(): double
uses
« interface »
DepartmentApprovalSystem
+ requestApproval(u: User): ApprovalStatus
+ getApprovalStatus(email: String): ApprovalStatus
Student
+ getHourlyRate(): double
Faculty
+ getHourlyRate(): double
Researcher
+ getHourlyRate(): double
Guest
+ getHourlyRate(): double
PaymentType
PaymentMethod
EquipmentStatus
PaymentStatus
ReservationStatus
AccountStatus
ApprovalStatus

Part III: Report on AI-Assistant Adaptation

Task 1 – Explain the Role of the AI Assistant

During Parts I and II of the project, we used ChatGPT (GPT-5.2) as a brainstorming and verification tool to help interpret the project requirements, refine actor and use-case identification, and review UML diagrams. The AI was mainly used to generate ideas and check completeness, but we did not rely blindly on its outputs. All suggestions were verified against lecture notes, assignment instructions, and our own reasoning before being included in the final design.

For use case diagrams, we asked prompts such as "Identify actors and use cases for a lab equipment reservation system." The AI suggested actors like User, Lab Manager, Head Lab Coordinator, Payment System, Equipment Sensor System, Department Approval System, and System Clock, along with use cases such as Register Account, Reserve Equipment, Modify Reservation, Cancel Reservation, and Pay Deposit. These suggestions helped us quickly brainstorm ideas, but we refined them using lecture concepts such as actor generalization and requirement mapping.

For activity and sequence diagrams, we used prompts about payment processing and reservation workflows. The AI helped outline logical steps, but we corrected method names, interaction order, and diagram structure to ensure they matched the project requirements.

For the class diagram, we asked the AI to list classes, attributes, and relationships. While the AI suggested many correct classes, some method signatures and UML relationships were incorrect or incomplete, so we refined them manually using lecture material.

Overall, the AI assistant was useful for brainstorming and checking completeness, but most design decisions were made manually. This ensured our final diagrams were accurate and aligned with proper software design practices.

Task2: Arguments and Resolution

During Parts I and II, there were multiple cases where our understanding of the requirements and UML standards differed from the AI assistant's suggestions. In general, the AI was useful for brainstorming, but several outputs contained inaccuracies (missing functionality, inconsistent naming, incorrect UML notation, or overly confident claims). We resolved these issues by cross-checking the project requirements, applying lecture rules for UML notation, and ensuring internal consistency across use case, activity, sequence, and class diagrams.

One disagreement occurred in how the AI initially framed use cases. The AI's early suggestions listed interactions in vague nouns (e.g., "Equipment Management," "Payment Processing") rather than use-case style verb phrases. We challenged this because use cases in UML are expected to describe actions/goals (verb phrases) and should be clearly traceable to requirements. We resolved this by rewriting them into concrete use cases such as Register Account, Pay Deposit, Modify Reservation, Cancel Reservation, and Extend Reservation, and

by keeping actor generalization (User → Student/Faculty/Researcher/Guest) to reflect shared behavior with different business rules.

A key disagreement involved the level of detail and the triggering logic for the deposit rule. The AI described deposit forfeiture in a generic way, but did not consistently connect it to the "first 20 minutes of the booking period" requirement and the automatic nature of the rule. We challenged this as incomplete, because the requirement implies a time-driven decision that should be modeled as a system event. We resolved this by explicitly including System Clock-triggered checks (reservation start time + 20-minute window) and branching to forfeit deposit vs. continue reservation. We also ensured the workflow included the sensor system's role in collecting usage data and that the activity flow matched the payment logic (deposit applied vs. forfeited).

In sequence diagram guidance, the AI sometimes proposed message orders and object responsibilities that did not match our final class design. For example, it suggested the "system" directly handling everything without clear object interactions, which can become inconsistent with a class diagram. We resolved this by aligning sequence messages with our class-level responsibilities: reservations interact with equipment availability checks, payments interact with the payment gateway, and automated checks (late arrival) are triggered by the system clock. We also corrected method names to be consistent across diagrams (e.g., using the same naming style for modify/cancel/extend operations).

The most significant disagreements occurred in the class diagram stage. The AI originally produced method signatures that were not Java-consistent (UML-like pseudo signatures). We challenged this because our deliverable requires a design that is implementable in Java and consistent with lecture conventions. We resolved this by converting signatures to Java-style types and parameters (e.g., using LocalDateTime consistently and defining parameters explicitly in methods).
Another disagreement involved unnecessary relationships. The AI initially implied strong links between administrative roles and reservations (e.g., implying LabManager directly interacts with Reservation). We rejected that because the requirements state Lab Managers manage equipment status/maintenance, not user bookings. We resolved this by keeping LabManager associated with Equipment and HeadLabCoordinator associated with LabManager account creation, while leaving reservations tied to users and equipment as the core booking model.

Across all tasks, disagreements were resolved by carefully reviewing the project requirements, applying the UML rules learned in lectures, and checking that all diagrams were consistent with each other. We compared the AI suggestions with the assignment description to confirm details such as the 20-minute deposit rule, the role of automated systems like the System Clock and Sensor System, and the correct responsibilities of actors such as Lab Manager and Head Lab Coordinator. We also ensured that use cases were written as clear actions, that sequence diagrams matched the class responsibilities, and that class diagram method signatures followed Java conventions. By correcting these issues and

verifying every change against lecture material and system requirements, we ensured that the final design was accurate, logically consistent, and not simply based on unverified AI output.

Task3: Verifying AI Responses

To verify the accuracy of AI assistant responses, we compared all suggestions against three sources: the project requirements, lecture material on UML and OOP design, and consistency across our own diagrams. The AI was mainly used for brainstorming, so every suggestion was reviewed before being included in the final work.

First, we checked AI outputs against the assignment requirements to ensure that important rules were included. For example, when the AI suggested activity diagram logic for payments, we verified that it correctly handled the 20-minute late arrival rule and deposit forfeiture. If details were missing, we updated the diagrams to match the requirements. Second, we verified UML correctness using lecture guidelines. In one case, the AI approved diagrams where "uses" relationships were drawn with inheritance-style arrows. After checking UML notation rules, we corrected them to dashed dependency arrows. This suggestion was modified to follow proper UML standards.

Third, we checked consistency across diagrams. If a sequence diagram showed a method, we ensured it existed in the class diagram. When the AI suggested non-Java-style method signatures, we corrected them to use proper types like LocalDateTime. These suggestions were also modified.

One example of an incorrect AI suggestion involved connecting Lab Manager directly to Reservation. After reviewing the requirements, we determined that Lab Managers only manage equipment, not bookings, so we removed that relationship. This suggestion was rejected.

Overall, AI responses were verified by checking requirements, UML rules, and diagram consistency. This process ensured the final design was accurate and not based on unverified AI output.

Task4: Application of AI-Suggested Content

One example of AI-suggested content that was used directly in our project was the identification of the main system actors. We used ChatGPT (GPT-5.2) to brainstorm possible actors based on the system requirements.

Original Prompt: "Identify actors for a lab equipment reservation system with features like user registration, equipment reservation, payments, sensors, and manager administration."

AI Output (excerpt used): "The actors are User, Lab Manager, Head Lab Coordinator, Payment Gateway, Equipment Sensor System, Department Approval System, and System Clock."

This suggestion was applied directly in Part I – Use Case Diagrams, where these actors appear in our diagrams and actor descriptions. The AI output was accepted without significant modification because it correctly matched the system requirements: each suggested actor represented a real entity interacting with the system, and no unnecessary actors were included. We verified this list against the assignment description and lecture examples before using it.

The AI suggestion was useful because it helped us quickly identify all required actors in a structured way. Since the result was accurate and consistent with the requirements, we used it directly in our final design.